증명 생성 (Proof Generation)

개요

증명 생성은 회로 witness를 입력으로 받아 영지식 증명을 생성하는 과정입니다. Scroll ZKEVM에서는 청크(Chunk) 단위로 증명을 생성하며, 이 증명들은 이후 배치(Batch)와 번들(Bundle)로 집계됩니다.

이 문서는 다음 내용을 다룹니다:

- Halo2 증명 시스템 사용
- 청크 증명 생성 프로세스
- Setup 및 Proving Key 생성
- 증명 검증 과정
- 성능 최적화 전략

증명 생성 파이프라인

전체 흐름

```
Block Data (Geth Trace)

↓

CircuitInputBuilder

↓

Witness Block

↓
```

```
SuperCircuit Instance
↓
[Setup Phase]
├─ Params Generation (KZG)
├─ Proving Key (PK)
└─ Verifying Key (VK)
↓
[Proving Phase]
├─ Circuit Synthesis
├─ Polynomial Commitment
└─ Proof Generation
↓
Chunk Proof (SNARK)
```

Halo2 증명 시스템

KZG 커밋먼트 스킴

Scroll ZKEVM은 KZG (Kate-Zaverucha-Goldberg) 다항식 커밋먼트 스킴을 사용합니다.

특징:

- 간결한 증명 크기 (상수 크기)
- 효율적인 검증 (단일 페어링 체크)
- 신뢰 설정(Trusted Setup) 필요

파라미터 구조:

```
pub struct ParamsKZG<E: Engine> {
pub k: u32, // 회로 크기 (2^k 행)
pub n: u64, // 2^k
pub g: Vec<E::G1Affine>, // G1 생성원들
pub g_lagrange: Vec<E::G1Affine>, // Lagrange 기저
pub g2: E::G2Affine, // G2 생성원
pub s_g2: E::G2Affine, // [s]₂ (신뢰 설정)
}
```

Proving Key와 Verifying Key

Proving Key (PK):

```
pub struct ProvingKey<C: CurveAffine> {
pub vk: VerifyingKey<C>, // Verifying Key 포함
pub l0: Polynomial<C::Scalar, Coeff>,
pub l_last: Polynomial<C::Scalar, Coeff>,
pub l_active_row: Polynomial<C::Scalar, Coeff>,
pub fixed_values: Vec<Polynomial<C::Scalar, LagrangeCoeff>>,
pub fixed_polys: Vec<Polynomial<C::Scalar, Coeff>>,
pub fixed_cosets: Vec<Polynomial<C::Scalar, ExtendedLagrangeCoeff>>,
pub permutation: permutation::ProvingKey<C>,
pub ev: Evaluator<C>,
```

```
}
```

Verifying Key (VK):

```
pub struct VerifyingKey<C: CurveAffine> {
pub domain: EvaluationDomain<C::Scalar>,
pub fixed_commitments: Vec<C>,
pub permutation: permutation::VerifyingKey<C>,
pub cs: ConstraintSystem<C::Scalar>,
pub cs_degree: usize,
pub transcript_repr: C::Scalar,
}
```

Setup Phase

1. 파라미터 생성

함수: `ParamsKZG::setup`

```
use halo2_proofs::poly::kzg::commitment::ParamsKZG;
use halo2curves::bn256::Bn256;

// k 값 결정 (회로 크기 = 2^k)
let k = 20; // 예: 2^20 = 1,048,576 행
```

```rust
// KZG 파라미터 생성 (신뢰 설정)
let params = ParamsKZG::<Bn256>::setup(k, rng);
```

k 값 결정:

```rust
// SuperCircuit의 필요한 행 수 계산
let (_, rows_needed) = SuperCircuit::min_num_rows_block(&block);

// unusable rows 고려
let k = log2_ceil(SuperCircuit::unusable_rows() + rows_needed);
```

신뢰 설정:

- 실제 프로덕션에서는 Powers of Tau 세레모니 결과 사용
- 테스트에서는 `setup` 함수로 생성

2. Proving Key 생성

함수: `keygen_pk`

```rust
use halo2_proofs::plonk::{keygen_vk, keygen_pk};

// 1. Verifying Key 생성
let vk = keygen_vk(&params, &circuit)?;

// 2. Proving Key 생성
```

```
let pk = keygen_pk(&params, vk, &circuit)?;
```

처리 과정:

1. **회로 합성 (Synthesis)**:

- 회로의 `synthesize` 메서드 호출
- 제약 조건 수집
- Fixed column 값 계산

2. **전처리 (Preprocessing)**:

- Fixed column 커밋먼트 생성
- Permutation argument 설정
- Lookup table 커밋먼트

3. **키 생성**:

- VK: 검증에 필요한 공개 정보
- PK: VK + 증명 생성에 필요한 추가 정보

3. 회로 크기 최적화

동적 크기 조정:

```
impl SuperCircuit {
pub fn min_num_rows_block(block: &Block) -> (usize, usize) {
let evm_rows = EvmCircuit::min_num_rows_block(block);
let keccak_rows = KeccakCircuit::min_num_rows_block(block);
let tx_rows = TxCircuit::min_num_rows_block(block);
// ... 기타 회로들
```

```
let max_rows = [
evm_rows,
keccak_rows,
tx_rows,
// ...
].iter().max().unwrap();
(*max_rows, *max_rows)
}
}
```

Proving Phase

1. 회로 인스턴스 생성

SuperCircuit 빌드:

```
// prover/src/lib.rs
pub fn build_super_circuit(
geth_data: GethData,
circuits_params: CircuitsParams,
) -> Result<(u32, SuperCircuit, Vec<Vec<Fr>>), Error> {
// BlockData 생성
let block_data = BlockData::new_from_geth_data_with_params(
geth_data.clone(),
```

```
circuits_params,
);
// CircuitInputBuilder 실행
let mut builder = block_data.new_circuit_input_builder();
builder.handle_block(
&geth_data.eth_block,
&geth_data.geth_traces,
)?;
// Witness Block 변환
let block = block_convert(&builder.block, &builder.code_db)?;
// SuperCircuit 생성
let circuit = SuperCircuit::new_from_block(&block);
// k 값 계산
let (_, rows_needed) = SuperCircuit::min_num_rows_block(&block);
let k = log2_ceil(SuperCircuit::unusable_rows() + rows_needed);
// Public Input 생성
let instance = circuit.instance();
Ok((k, circuit, instance))
}
```

2. 증명 생성

함수: `create_proof`

```rust
use halo2_proofs::plonk::create_proof;
use halo2_proofs::transcript::{
Blake2bWrite, Challenge255, TranscriptWriterBuffer,
};


pub fn gen_proof(
params: &ParamsKZG<Bn256>,
pk: &ProvingKey<G1Affine>,
circuit: SuperCircuit,
instances: Vec<Vec<Fr>>,
rng: impl RngCore,
) -> Result<Vec<u8>, Error> {
// 트랜스크립트 생성
let mut transcript = Blake2bWrite::<_, G1Affine, Challenge255<_>>::init(vec![]
// 증명 생성
create_proof::<KZGCommitmentScheme<Bn256>, ProverGWC<_>, _, _, _, _>(
params,
pk,
&[circuit],
&[&instances.iter().map(|v| &v[..]).collect::<Vec<_>>()],
rng,
&mut transcript,
```

```
)?;

// 증명 바이트 추출

Ok(transcript.finalize())

}
```

증명 생성 단계:

1. **Commitment Phase**:
   - Advice column 값을 다항식으로 변환
   - 다항식을 KZG로 커밋
   - 커밋먼트를 트랜스크립트에 추가
2. **Challenge Generation**:
   - Fiat-Shamir 변환으로 challenge 생성
   - 트랜스크립트 상태 기반
3. **Evaluation Phase**:
   - Challenge 점에서 다항식 평가
   - 평가값을 트랜스크립트에 추가
4. **Opening Phase**:
   - KZG opening proof 생성
   - 다항식이 주장한 값으로 평가됨을 증명

3. 증명 구조

SNARK 구조:

```
pub struct Snark {
```

```rust
pub protocol: Protocol, // 회로 메타데이터
pub instances: Vec<Vec<Fr>>, // Public input
pub proof: Vec<u8>, // 증명 바이트
}

pub struct Protocol {
pub domain: Domain,
pub preprocessed: Vec<C>, // Fixed column 커밋먼트
pub num_instance: Vec<usize>, // Instance column 수
pub num_witness: Vec<usize>, // Advice column 수
pub num_challenge: Vec<usize>, // Challenge 수
pub evaluations: Vec<Query>, // 평가 쿼리
pub queries: Vec<Query>, // 다항식 쿼리
pub quotient: QuotientPolynomial,
pub transcript_initial_state: Option<C::Scalar>,
pub instance_committing_key: Option<InstanceCommittingKey<C>>,
pub linearization: Option<LinearizationStrategy>,
pub accumulator_indices: Vec<Vec<(usize, usize)>>,
}
```

청크 증명 생성

ChunkProof 구조

```rust
// aggregator/src/chunk.rs
pub struct ChunkProof {
pub chunk_info: ChunkInfo, // 청크 메타데이터
pub snark: Snark, // 증명
}

pub struct ChunkInfo {
pub chain_id: u64,
pub prev_state_root: H256,
pub post_state_root: H256,
pub withdraw_root: H256,
pub data_hash: H256,
pub tx_bytes: Vec<u8>,
pub is_padding: bool,
}
```

청크 증명 생성 프로세스

```rust
// prover/src/chunk_prover.rs
pub fn gen_chunk_proof(
params: &ParamsKZG<Bn256>,
pk: &ProvingKey<G1Affine>,
chunk_trace: ChunkTrace,
```

```
rng: impl RngCore,
) -> Result<ChunkProof, Error> {
// 1. SuperCircuit 빌드
let (k, circuit, instances) = build_super_circuit(
chunk_trace.geth_data,
chunk_trace.circuits_params,
)?;
// 2. 파라미터 크기 확인
assert!(k <= params.k(), "Circuit too large for params");
// 3. 증명 생성
let  proof_bytes = gen_proof(params, pk, circuit, instances.clone(), rng)?;
// 4. SNARK 구성
let  protocol = compile_protocol(params, pk.get_vk())?;
let  snark = Snark {
protocol,
instances,
proof: proof_bytes,
};
// 5. ChunkInfo 생성
let  chunk_info = ChunkInfo::from_block_traces(&chunk_trace.block_traces)?;
Ok(ChunkProof { chunk_info, snark })
}
```

병렬 청크 증명 생성

```
use rayon::prelude::*;

pub fn gen_chunk_proofs_parallel(
params: &ParamsKZG<Bn256>,
pk: &ProvingKey<G1Affine>,
chunk_traces: Vec<ChunkTrace>,
) -> Result<Vec<ChunkProof>, Error> {
chunk_traces
.par_iter()
.map(|chunk_trace| {
let rng = thread_rng();
gen_chunk_proof(params, pk, chunk_trace.clone(), rng)
})
.collect()
}
```

증명 검증

검증 프로세스

함수: `verify_proof`

```rust
use halo2_proofs::plonk::verify_proof;
use halo2_proofs::transcript::{
Blake2bRead, Challenge255, TranscriptReadBuffer,
};


pub fn verify_snark(
params: &ParamsKZG<Bn256>,
vk: &VerifyingKey<G1Affine>,
snark: &Snark,
) -> Result<(), Error> {
// 트랜스크립트 생성
let mut transcript = Blake2bRead::<_, G1Affine, Challenge255<_>>::init(
&snark.proof[..]
);
// 증명 검증
verify_proof::<KZGCommitmentScheme<Bn256>, VerifierGWC<_>, _, _, _>(
params,
vk,
SingleStrategy::new(params),
&[&snark.instances.iter().map(|v| &v[..]).collect::<Vec<_>>()],
&mut transcript,
)?;
```

```
Ok(())
}
```

검증 단계:

1. **커밋먼트 읽기**:
   - 증명에서 다항식 커밋먼트 추출
   - 트랜스크립트 상태 업데이트
2. **Challenge 재생성**:
   - Prover와 동일한 challenge 생성
   - Fiat-Shamir 변환 사용
3. **평가 검증**:
   - 주장된 평가값 확인
   - Opening proof 검증
4. **페어링 체크**:
   - KZG opening의 최종 검증
   - $e(\text{commitment}, [1]_2) = e(\text{proof}, [s]_2) \cdot e([\text{evaluation}]_1, [1]_2)$

Public Input 검증

```
pub fn verify_chunk_public_input(
chunk_proof: &ChunkProof,
) -> Result<(), Error> {
// 1. ChunkInfo에서 예상 PI 해시 계산
let expected_pi_hash = chunk_proof.chunk_info.public_input_hash();
```

```
// 2. SNARK 인스턴스에서 실제 PI 해시 추출
let actual_pi_hash = extract_pi_hash_from_instances(
&chunk_proof.snark.instances
)?;
// 3. 일치 확인
if expected_pi_hash != actual_pi_hash {
return Err(Error::PublicInputMismatch);
}
Ok(())
}
```

성능 최적화

1. 회로 크기 최적화

동적 패딩:

```
impl SuperCircuit {
fn synthesize_sub(
&self,
config: &Self::Config,
challenges: &Challenges<Value<Fr>>,
layouter: &mut impl Layouter<Fr>,
) -> Result<(), Error> {
```

```
// 각 회로를 필요한 만큼만 할당
self.evm_circuit.synthesize_sub(config, challenges, layouter)?;
self.state_circuit.synthesize_sub(config, challenges, layouter)?;
// ...
// 남은 공간을 패딩으로 채움
self.pad_to_capacity(layouter)?;
Ok(())
}
}
```

2. 메모리 최적화

스트리밍 증명 생성:

```
// 큰 witness를 메모리에 모두 로드하지 않고 스트리밍
pub fn gen_proof_streaming(
params: &ParamsKZG<Bn256>,
pk: &ProvingKey<G1Affine>,
witness_stream: impl Iterator<Item = WitnessChunk>,
) -> Result<Vec<u8>, Error> {
// 구현...
}
```

3. GPU 가속

GPU 기반 MSM (Multi-Scalar Multiplication):

```
#[cfg(feature = "gpu")]
use halo2_proofs::poly::commitment::MSM;

pub fn gen_proof_gpu(
params: &ParamsKZG<Bn256>,
pk: &ProvingKey<G1Affine>,
circuit: SuperCircuit,
instances: Vec<Vec<Fr>>,
) -> Result<Vec<u8>, Error> {
// GPU 가속 MSM 사용
let msm = MSM::new_gpu()?;
// 증명 생성 (GPU 가속)
// ...
}
```

4. 병렬화

멀티스레드 증명 생성:

```
use rayon::prelude::*;

pub fn gen_proofs_parallel(
params: &ParamsKZG<Bn256>,
```

```
pk: &ProvingKey<G1Affine>,

circuits: Vec<SuperCircuit>,

) -> Result<Vec<Vec<u8>>, Error> {

circuits

.par_iter()

.map(|circuit| {

let  instances = circuit.instance();

let  rng = thread_rng();

gen_proof(params, pk, circuit.clone(), instances, rng)

})

.collect()

}
```

증명 크기 및 시간

일반적인 메트릭

청크 증명:

- 증명 크기: ~200-300 KB
- 생성 시간: ~30-60초 (CPU), ~5-10초 (GPU)
- 검증 시간: ~10-20ms

회로 크기 (k 값):

- 작은 블록: k = 18-19 (262K-524K 행)

- 중간 블록: k = 20-21 (1M-2M 행)
- 큰 블록: k = 22-23 (4M-8M 행)

크기 결정 요인

1. **트랜잭션 수**: 더 많은 트랜잭션 = 더 큰 회로
2. **Opcode 복잡도**: 복잡한 연산 (CALL, CREATE) = 더 많은 행
3. **메모리/스토리지 사용**: 많은 RW 연산 = 더 큰 State Circuit
4. **Keccak 해시**: 많은 해시 = 더 큰 Keccak Circuit

에러 처리

일반적인 에러

1. 회로 크기 초과:

```
if k > params.k() {
return Err(Error::CircuitTooLarge {
required: k,
available: params.k(),
});
}
```

2. 제약 조건 실패:

```
// 증명 생성 중 제약 조건 위반 감지
match create_proof(...) {
```

```
Err(halo2_proofs::plonk::Error::ConstraintSystemFailure) => {
// 디버깅 정보 수집
let failing_constraints = debug_circuit(&circuit)?;
Err(Error::ConstraintViolation(failing_constraints))
}
Err(e) => Err(e.into()),
Ok(proof) => Ok(proof),
}
```

3. Public Input 불일치:

```
pub fn validate_public_input(
circuit: &SuperCircuit,
expected_pi: &[Fr],
) -> Result<(), Error> {
let actual_pi = circuit.instance();
if actual_pi != expected_pi {
return Err(Error::PublicInputMismatch {
expected: expected_pi.to_vec(),
actual: actual_pi,
});
}
Ok(())
```

```
}
```

디버깅 및 테스트

MockProver 사용

```
use halo2_proofs::dev::MockProver;

#[test]
fn test_circuit_constraints() {
let circuit = SuperCircuit::new_from_block(&block);
let instances = circuit.instance();
// MockProver로 제약 조건 검증 (증명 생성 없이)
let prover = MockProver::run(k, &circuit, instances).unwrap();
// 제약 조건 위반 확인
assert_eq!(prover.verify(), Ok(()));
}
```

회로 시각화

```
use plotters::prelude::*;

pub fn visualize_circuit(
circuit: &SuperCircuit,
```

```
output_path: &str,
) -> Result<(), Error> {
let  root = BitMapBackend::new(output_path, (1024, 768))
.into_drawing_area();
root.fill(&WHITE)?;
// 회로 레이아웃 그리기
halo2_proofs::dev::CircuitLayout::default()
.render(k, circuit, &root)?;
Ok(())
}
```

실제 사용 예시

단일 청크 증명 생성

```
use prover::{build_super_circuit, gen_proof};

// 1. Geth 데이터 로드
let  geth_data = load_geth_data("block_trace.json")?;

// 2. 회로 파라미터 설정
let  circuits_params = CircuitsParams {
max_txs: 100,
```

```
max_calldata: 200_000,
max_rws: 1_000_000,
// ...
};

// 3. SuperCircuit 빌드
let (k, circuit, instances) = build_super_circuit(
geth_data,
circuits_params,
)?;

// 4. 파라미터 및 키 로드
let  params = load_params(k)?;
let  pk = load_proving_key(&params, &circuit)?;

// 5. 증명 생성
let  rng = thread_rng();
let  proof = gen_proof(&params, &pk, circuit, instances, rng)?;

// 6. 증명 저장
save_proof("chunk_proof.bin", &proof)?;
```

배치 증명 생성 파이프라인

```
// 1. 여러 청크 증명 생성
let  chunk_proofs = gen_chunk_proofs_parallel(
&params,
&chunk_pk,
chunk_traces,
)?;


// 2. 청크 증명 압축
let  compressed_proofs = chunk_proofs
.iter()
.map(|proof| compress_proof(&params, &compression_pk, proof))
.collect::<Result<Vec<_>, _>>()?;


// 3. 배치 증명 생성
let  batch_proof = gen_batch_proof(
&params,
&batch_pk,
compressed_proofs,
)?;


// 4. 배치 증명 압축
let  compressed_batch = compress_proof(
```

```
&params,
&compression_pk,
&batch_proof,
)?;
```

관련 문서

- 청킹 전략: 멀티 블록 청킹 메커니즘
- 증명 집계: 증명 집계 프로세스
- 데이터 흐름: 전체 데이터 변환 과정
- Halo2 통합: Halo2 사용 방법

참고 자료

코드 위치:

- `prover/src/lib.rs` : 증명 생성 메인 로직
- `prover/src/chunk_prover.rs` : 청크 증명 생성
- `aggregator/src/chunk.rs` : ChunkInfo 및 ChunkProof 구조
- `zkevm-circuits/src/super_circuit.rs` : SuperCircuit 구현

외부 의존성:

- `halo2_proofs` : Halo2 증명 시스템
- `halo2curves` : 타원곡선 구현 (BN256)
- `rayon` : 병렬 처리
- `rand` : 난수 생성

요약

증명 생성은 ZKEVM의 핵심 프로세스로, 다음 단계로 구성됩니다:

1. **Setup**: KZG 파라미터 및 Proving/Verifying Key 생성
2. **Circuit Building**: Witness 데이터로 회로 인스턴스 생성
3. **Proof Generation**: Halo2를 사용한 SNARK 생성
4. **Verification**: 증명의 유효성 검증
5. **Optimization**: 병렬화, GPU 가속, 메모리 최적화

생성된 청크 증명은 이후 압축 및 집계 단계를 거쳐 최종 번들 증명으로 변환됩니다.